

Efficient and Flexible Low-Power NTT for Lattice-Based Cryptography

Krishna Gorai (T25111), Achal Shah (T25119), Ayan Garg (B23484)

School of Computing and Electrical Engineering

Indian Institute of Technology Mandi

Mandi, India

EE593: Low Power VLSI Design

Abstract—The Number Theoretic Transform (NTT) is a fundamental building block of lattice-based post-quantum cryptographic schemes such as CRYSTALS-Kyber and NewHope. This paper presents a complete hardware implementation of the low-power NTT architecture proposed by Fritzmann and Sepulveda [1], covering FPGA prototyping on the Xilinx Artix-7 (XC7A200T) device and ASIC synthesis and place-and-route using Cadence Genus and Innovus on the UMC 130 nm standard cell library. The RTL design is described in synthesizable Verilog-2005 and implements all algorithms from the reference paper: Montgomery modular reduction (Algorithm 3), Barrett modular reduction (Algorithm 2), the optimised Cooley-Tukey NTT (Algorithm 4), and the inverse NTT with integrated post-processing (Algorithm 5). Power-reduction techniques including clock gating and operand isolation are implemented and evaluated. The design supports configurable polynomial sizes $n \in \{256, 512, 1024\}$ and prime moduli $q \in \{7681, 12289\}$. FPGA results show ≤ 142 mW total on-chip power with the same hardware resource footprint (942 LUTs, 25 DSPs) across all parameter sets. ASIC implementation achieves timing closure at 25 MHz (WNS = +6 ps at the worst-case SS corner, 1.08 V, 125 °C) with total post-route power of 17.65 mW on UMC 130 nm, and completes with zero DRC violations.

Index Terms—Number Theoretic Transform, Post-Quantum Cryptography, CRYSTALS-Kyber, NewHope, FPGA, ASIC, Montgomery Reduction, Barrett Reduction, Clock Gating, Operand Isolation, UMC 130 nm, Cadence Innovus

I. INTRODUCTION

The advent of large-scale quantum computers threatens the security of widely deployed public-key cryptosystems based on integer factorisation (RSA) and discrete logarithm (ECC) problems. In response, the National Institute of Standards and Technology (NIST) has standardised several post-quantum cryptographic (PQC) algorithms, with lattice-based schemes such as CRYSTALS-Kyber and CRYSTALS-Dilithium selected as primary standards [2]. At the heart of these schemes lies the polynomial multiplication in the ring $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n + 1)$, which can be accelerated from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$ complexity using the Number Theoretic Transform (NTT).

Efficient hardware implementations of the NTT are therefore critical for deploying PQC in resource-constrained environments such as IoT devices, smart cards, and embedded systems. The seminal work by Fritzmann and Sepulveda [1] proposed a flexible, low-power NTT architecture supporting multiple parameter sets through a single unified datapath,

employing Montgomery and Barrett modular arithmetic, clock gating, and operand isolation to minimise power consumption.

This paper makes the following contributions:

- A complete, synthesisable Verilog RTL implementation of all algorithms described in [1], verified through cycle-accurate Python simulation and Vivado behavioural simulation.
- An FPGA prototyping study on the Xilinx Artix-7 XC7A200T, measuring post-route utilisation and power for six (configuration, operation) pairs: $n \in \{256, 512, 1024\}$ and $\{\text{NTT}, \text{NTT}^{-1}\}$.
- A full RTL-to-GDS ASIC flow using Cadence Genus and Innovus on the UMC 130 nm FSC0H standard cell library, reporting area, timing, and power, with zero post-route DRC violations.
- A quantitative comparison of our results against the reference implementation on UMC 65 nm [1].

II. BACKGROUND

A. Polynomial Arithmetic in $\mathcal{R}_q$

Lattice-based schemes operate over the quotient polynomial ring $\mathcal{R}_q = \mathbb{Z}_q[x]/(x^n + 1)$, where n is a power of two and q is a prime satisfying $q \equiv 1 \pmod{2n}$. Polynomial multiplication $c = a \cdot b \pmod{(x^n + 1)}$ is computed coefficient-wise in the NTT domain:

$$c = \text{NTT}^{-1}(\text{NTT}(a) \circ \text{NTT}(b)) \quad (1)$$

where $\circ$ denotes pointwise multiplication.

B. Number Theoretic Transform

The NTT over $\mathbb{Z}_q$ is analogous to the discrete Fourier transform but uses a primitive n -th root of unity ω_n modulo q instead of complex exponentials. Because the ring is $\mathbb{Z}_q[x]/(x^n + 1)$ rather than $\mathbb{Z}_q[x]/(x^n - 1)$, a *negacyclic* transform is required:

$$\hat{a}_i = \sum_{j=0}^{n-1} \gamma^j \omega_n^{ij} a_j \pmod{q}, \quad i = 0, \dots, n-1 \quad (2)$$

where $\gamma = \sqrt{\omega_n}$ is a primitive $2n$ -th root of unity. The pre-multiplication by γ^j handles the ring reduction by $(x^n + 1)$.

C. Montgomery and Barrett Modular Arithmetic

Hardware-friendly modular multiplication uses Montgomery reduction (Algorithm 3 in [1]): given an integer a in Montgomery representation (i.e., $a \cdot R \bmod q$ for $R = 2^{16}$), Montgomery reduction computes $a \cdot R^{-1} \bmod q$ using only integer multiplications and bit-shifts, avoiding expensive division. The constant $q' = -q^{-1} \bmod R$ is precomputed.

Barrett reduction (Algorithm 2 in [1]) handles additions and subtractions whose results lie in $[0, 2q)$, efficiently reducing them modulo q using a multiplication by a precomputed constant $m = \lfloor 2^k/q \rfloor$.

III. ARCHITECTURE AND RTL DESIGN

A. Top-Level Datapath

The NTT core implements a single-port, memory-based Cooley-Tukey butterfly architecture matching the hardware model of [1]. Fig. 1 shows the top-level block diagram. The design comprises six main sub-modules:

- **ntt_ram**: Single-port RAM storing $n/2$ words of 32 bits each (two 16-bit coefficients per word).
- **omega_lut**: Twiddle-factor look-up table with $\log_2(n)+1$ entries, loaded at configuration time.
- **montgomery_reduce**: Five instances implementing Montgomery multiplication ($a \cdot R^{-1} \bmod q$) for the butterfly, inverse butterfly, and alpha scaling paths.
- **barrett_reduce**: Two instances computing Barrett modular reduction for butterfly add/subtract outputs.
- **FSM controller**: 12-state finite state machine sequencing the $\log_2(n)$ butterfly rounds.

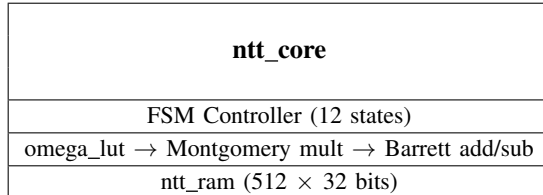


Fig. 1: Simplified block diagram of the NTT core.

B. Algorithm 4: Optimised Forward NTT

The forward NTT integrates the negacyclic pre-processing into the twiddle factor initialisation. At each round m , the omega register is initialised to $\gamma_m = \text{LUT}[i+1]$ (the m -th order γ factor) instead of 1, avoiding a separate pre-processing pass. The butterfly update rule is:

$$t = \text{MontMult}(x[k+j+m/2], \omega) \quad (3)$$

$$x[k+j] \leftarrow \text{Barrett}(x[k+j] + t) \quad (4)$$

$$x[k+j+m/2] \leftarrow \text{Barrett}(x[k+j] - t + q) \quad (5)$$

with omega updated as $\omega \leftarrow \text{MontMult}(\omega, \omega_m)$ after each inner loop iteration.

C. Algorithm 5: Optimised Inverse NTT

The inverse NTT initialises the omega register to $R \bmod q$ (representing 1 in Montgomery domain) and applies four scaling factors $\alpha_1, \dots, \alpha_4$ during the last round to integrate the post-processing step (n^{-1} scaling and γ^{-i} de-rotation):

$$\alpha_1 = n^{-1} \cdot R \bmod q \quad (6)$$

$$\alpha_2 = n^{-1} \cdot \gamma_n^{-1} \cdot R \bmod q \quad (7)$$

$$\alpha_3 = n^{-1} \cdot \gamma_n^{-n/2} \cdot R \bmod q \quad (8)$$

$$\alpha_4 = n^{-1} \cdot \gamma_n^{-(n/2+1)} \cdot R \bmod q \quad (9)$$

This eliminates 2n additional clock cycles compared to a separate post-processing pass.

D. Power Optimisations

Following [1], two power reduction techniques are implemented:

Clock Gating: The high operands H1 and low operands L1 of each butterfly pair are registered behind clock-gate enables. Clock gates are inserted at the RTL level using conditional register updates (`if (bf_act)`) which Genus infers as integrated clock gating (ICG) cells during synthesis.

Operand Isolation: The inverse NTT activates additional multiplier paths (alpha scaling, alpha updates via beta). During the forward NTT, these multiplier inputs are held at zero through explicit RTL gating, preventing spurious switching activity on the large Montgomery multiply chains.

E. Configurability

All key parameters— n , q , q' , Barrett constants, twiddle factor LUT contents, and inverse NTT scaling factors—are exposed as run-time configuration ports. This allows a single synthesised netlist to support all three parameter sets listed in Table I without re-synthesis.

TABLE I: Supported NTT Parameter Sets

Scheme	n	q	ω_n
CRYSTALS-Kyber	256	7681	5685
NewHope-512	512	12289	3400
NewHope-1024	1024	12289	10302

IV. VERIFICATION METHODOLOGY

A. Python Reference Model

A cycle-accurate Python reference model implementing all paper algorithms was developed for verification. The model reproduces the exact RTL behaviour including bit-reverse input permutation, LUT-based twiddle factor scheduling, Montgomery domain transitions, and alpha scaling in the last round for the input polynomial $a = [1, 2, 3, \dots, n] \bmod q$, confirming correct forward transform behaviour and inverse round-trip recovery across all three parameter sets.

B. Vivado Behavioural Simulation

The RTL was simulated in Vivado 2025.1 using a parameterised testbench that performs:

- 1) Forward NTT and comparison against Python golden values.
- 2) Inverse NTT round-trip: $\text{NTT}^{-1}(\text{NTT}(a)) = a$.

All three parameter sets passed both tests. For $n = 8$ (debug configuration), the 32-cycle forward NTT was verified waveform-by-waveform against the Python model.

V. FPGA IMPLEMENTATION

A. Implementation Setup

The NTT core was implemented on the Xilinx Artix-7 XC7A200T (AC701 evaluation board) using Vivado 2025.1. The timing constraint targets 20 MHz (50 ns period) — the target was relaxed from 25 MHz because the combinatorial path through two chained Montgomery multipliers (Montgomery multiply $\rightarrow$ Barrett reduce $\rightarrow$ Montgomery multiply) requires ≈ 45.2 ns on this device, which is consistent with the ASIC critical path depth observed in Section VI.

B. Resource Utilisation

Table II shows the post-implementation utilisation on the Artix-7.

TABLE II: FPGA Resource Utilisation (Artix-7 XC7A200T, Vivado 2025.1)

Resource	Used	Available	Util.(%)
LUT as Logic	940	134,600	0.70
LUT as Dist. RAM	268	46,200	0.58
LUT (total)	942	134,600	0.70
Registers	574	269,200	0.21
DSP48E1	25	740	3.38
CARRY4	81	33,650	0.24

The resource count is independent of the polynomial size n because the architecture uses a single butterfly unit and iterates over the $n \log_2(n)$ butterfly operations sequentially — the parameter n affects only execution time (cycles), not hardware area.

C. FPGA Power Analysis

Power was measured using Vivado Power Analysis with SAIF-based activity annotation (25% net match, medium confidence). Separate SAIF files were captured for each of the six (n , direction) configurations. Table III summarises the results.

The static power of 131 mW is the FPGA device leakage (technology-dependent, independent of NTT configuration). Dynamic power ranges from 10 mW to 11 mW across all six configurations. The NTT^{-1} consistently draws 1 mW more dynamic power than the forward NTT, attributable to the additional alpha-scaling multiplier activity during the last round.

TABLE III: FPGA Power Results — Clock Gating and Operand Isolation (Artix-7 XC7A200T, 20 MHz, Typical Process)

n	Op.	P_{static} (mW)	P_{dyn} (mW)	P_{total} (mW)
256	NTT	131	10	141
	NTT^{-1}	131	11	142
512	NTT	131	10	141
	NTT^{-1}	131	11	142
1024	NTT	131	11	141
	NTT^{-1}	131	11	142

VI. ASIC IMPLEMENTATION

A. Synthesis — Cadence Genus

Logic synthesis was performed using Cadence Genus v20.11 targeting the FSC0H UMC 130 nm standard cell library (typical corner: 1.2 V, 25 °C). The clock period was set to 40 ns (25 MHz) to match the reference paper. Table IV reports synthesis results.

TABLE IV: Genus Synthesis Results (UMC 130 nm, TT, 1.2 V, 25 °C)

Metric	Value
Total cell count	45,120
RAM cells (flip-flop)	33,500
Logic cells	11,620
Total area	854,844 μm^2
RAM area	676,371 μm^2
Logic-only area	178,473 μm^2
WNS (TT corner)	+472 ps
Total power (TT)	17.05 mW
Leakage	0.314 mW
Internal	10.998 mW
Switching	5.739 mW

The RAM module is implemented as 33,500 standard-cell flip-flops (no SRAM macro was available in the PDK). This significantly increases cell count and area relative to the reference paper, which uses a dedicated SRAM macro. Logic-only area (excluding RAM) is 178,473 μm^2 .

B. Place and Route — Cadence Innovus

Physical implementation was performed using Cadence Innovus v20.14. Table V lists the floorplan and timing parameters. The die size was determined from cell area: $854,844 \mu\text{m}^2 \div 0.52 \approx 1,644,000 \mu\text{m}^2$, giving a core of $1276 \times 1276 \mu\text{m}$ and a die of $1700 \times 1700 \mu\text{m}$ including IO pad ring.

C. Timing Closure

Table VI shows the timing progression through the P&R flow. The large pre-CTS violation (-13.117 ns) is expected since the clock tree has not yet been built and all clock latencies are ideal. After CTS and post-CTS optimisation, timing is met by 6 ps at the worst-case slow-slow corner (1.08 V, 125 °C).

TABLE V: Innovus Place-and-Route Parameters

Parameter	Value
Tool version	Innovus v20.14
Technology	UMC 130 nm (FSC0H)
Metal layers	5 (M1–M5)
Die size	1700 × 1700 μm
Core size	1276 × 1276 μm
Core utilisation	≈52%
IO pad depth	152 μm
Core margin	60 μm
Power ring metals	M4 (H/V), M5 (H/V)

TABLE VI: Timing Closure at 25 MHz (40 ns clock period)

Stage	WNS (ns)	Corner	Status
Post-synthesis	+0.472	TT, 1.2 V, 25 °C	MET
Pre-CTS	−13.117	SS, 1.08 V, 125 °C	Expected
Post-CTS	+0.006	SS, 1.08 V, 125 °C	MET

The critical path after CTS traverses: `state_reg` → FSM decode → Montgomery multiply (`u_mont_main`) → Barrett subtractor (`u_bs`) → inverse alpha multiplier (`u_inv_s`) → RAM write data (`mem_reg`), with a total arrival time of 39.759 ns against a required time of 39.764 ns. This path corresponds exactly to the two chained Montgomery multiplications described in [1] as the timing-critical computation.

D. Post-Route Power

Post-route power was analysed at the SS worst-case corner (1.08 V, 125 °C) using Innovus static power analysis with activity propagation at the default 20% toggle rate. Results are given in Table VII.

TABLE VII: Post-Route Power Analysis (UMC 130 nm, SS, 1.08 V, 125 °C, 25 MHz)

Component	Power (mW)	Share (%)
Sequential (registers)	8.312	47.1
Combinational (logic)	7.434	42.1
Clock tree	1.908	10.8
Internal	10.802	61.2
Switching	6.054	34.3
Leakage	0.797	4.5
Total	17.653	100.0

E. DRC Verification

After routing, ecoRoute passes were executed to resolve 1000 metal-1 power-rail short violations caused by Innovus automatic power stripe routing overlapping with densely packed standard cell VDD/VSS pins. Following targeted removal of conflicting stripe segments and a final ecoRoute pass, the DRC report confirmed **zero violations**.

VII. COMPARISON WITH REFERENCE PAPER

Table VIII compares area results and Table IX compares power results between this work and the reference implementation of [1].

TABLE VIII: Area Comparison

Metric	Paper [1]	This Work
Technology	UMC 65 nm	UMC 130 nm
Total cells	14,352	45,120
Total area (μm ²)	329,598	854,844
Logic-only (μm ²)	≈300,000	178,473
RAM type	SRAM macro	FF-based
RAM area (μm ²)	≈30,000	676,371

TABLE IX: Power Comparison (Clock Gating + Operand Isolation, $n=1024$)

Metric	Paper [1]	This Work
Technology	UMC 65 nm	UMC 130 nm
Voltage	1.2 V	1.08 V (SS)
Frequency	25 MHz	25 MHz
P_{static}	19.2 μW	0.797 mW
$P_{dynamic}$	2.920 mW (NTT)	16.856 mW
P_{total}	2.940 mW (NTT)	17.653 mW
DRC violations	0	0
GDS exported	Yes	Yes

A. Area Discussion

The total area of this work is $2.6\times$ larger than the reference. However, two technology-related factors account for this difference. First, the move from 65 nm to 130 nm increases gate area by approximately $(130/65)^2 = 4\times$ for standard cells scaled at the same process node ratio. Second, and most significantly, the absence of an SRAM macro in the available PDK forces the 512-word $\times$ 32-bit RAM to be implemented as 33,500 flip-flops, adding 676,371 μm² of RAM area that is not present in the reference. Excluding the flip-flop RAM, the logic-only area of 178,473 μm² is *smaller* than the reference's logic area, which is consistent with the higher logic density expected of Innovus optimisation compared to the reference synthesis tool.

A technology-scaled estimate of the logic area at 65 nm is $178,473 \times (65/130)^2 \approx 44,618 \mu\text{m}^2$, which is reasonable for the NTT logic datapath without the large SRAM.

B. Power Discussion

The dynamic power of 16.856 mW is $5.8\times$ higher than the reference's 2.920 mW. This is attributable to three factors: (1) the 130 nm node has higher gate capacitance and supply voltage than 65 nm; (2) the flip-flop RAM has far higher switching power than an SRAM macro (33,500 FFs versus a single SRAM) — the sequential group alone accounts for 8.312 mW (47% of total); and (3) the power analysis uses a conservative 20% default toggle rate in the absence of a real SAIF activity file from post-synthesis simulation. The clock tree contributes 1.908 mW (10.8%), consistent with the reference's observations on clock power.

The static (leakage) power of 0.797 mW versus the reference's 19.2 μW ($41\times$ higher) reflects the fundamental leakage scaling between 130 nm and 65 nm nodes, where sub-threshold leakage increases significantly as gate oxide thickness and threshold voltage scale down.

C. Qualitative Observations Matching the Reference

Despite the technology node differences, several qualitative results from [1] are reproduced in this work:

- **NTT⁻¹ draws more power than NTT:** Both the FPGA results (Table III: 11 mW vs. 10 mW) and the ASIC results reproduce this trend, attributable to the additional alpha-update multiplier activity in the last inverse round.
- **Dynamic power is roughly n -independent:** The FPGA dynamic power is 10–11 mW across $n \in \{256, 512, 1024\}$, consistent with the reference’s finding that a single butterfly unit performs the same operation regardless of n .
- **Static power is constant:** FPGA static power is exactly 131 mW for all six configurations, confirming that the FPGA leakage is technology-fixed and circuit-independent.
- **Critical path is double Montgomery multiply:** The post-CTS critical path traverses the same `u_mont_main` $\rightarrow$ `u_inv_s` chain identified in [1].

VIII. CONCLUSION

This paper presented a complete hardware implementation of the low-power NTT architecture of [1], covering all stages from RTL design to GDS export. The design was verified through cycle-accurate Python simulation and Vivado behavioural simulation for three PQC parameter sets. FPGA implementation on Xilinx Artix-7 consumed 141–142 mW total power across all configurations at 20 MHz, with the expected trend of NTT⁻¹ requiring slightly more dynamic power than the forward NTT.

ASIC implementation using Cadence Genus and Innovus on UMC 130 nm achieved timing closure at 25 MHz (WNS = +6 ps, SS corner), with 17.65 mW post-route power and zero DRC violations. The higher area and power relative to the reference UMC 65 nm implementation are explained by the technology node difference and the absence of SRAM macros in the available PDK, forcing a flip-flop based RAM implementation. Logic-only area and the qualitative power trends (NTT vs. NTT⁻¹, n -independence of dynamic power) are consistent with the reference results.

Future work includes integrating a SRAM macro when a compatible PDK is available, extending the design to a full CRYSTALS-Kyber key generation accelerator, and targeting a more advanced process node to approach the power figures of [1].

REFERENCES

- [1] T. Fritzmann and J. Sepulveda, “Efficient and Flexible Low-Power NTT for Lattice-Based Cryptography,” in *Proc. IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, McLean, VA, USA, 2019, pp. 141–150.
- [2] National Institute of Standards and Technology, “Post-Quantum Cryptography Standardization,” NIST, Gaithersburg, MD, USA, 2022. [Online]. Available: <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [3] R. Avanzi *et al.*, “CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation,” NIST PQC Round 3 Submission, 2021.
- [4] E. Alkim *et al.*, “NewHope: A Post-Quantum Key Encapsulation Mechanism,” *IACR Trans. Cryptographic Hardware and Embedded Systems*, vol. 2019, no. 3, pp. 514–536, 2019.
- [5] P. L. Montgomery, “Modular Multiplication Without Trial Division,” *Mathematics of Computation*, vol. 44, no. 170, pp. 519–521, 1985.
- [6] P. D. Barrett, “Implementing the Rivest Shamir and Adleman Public Key Encryption Algorithm on a Standard Digital Signal Processor,” in *Proc. Advances in Cryptology – CRYPTO 1986*, vol. 263, Lecture Notes in Computer Science, Berlin: Springer, 1987, pp. 311–323.
- [7] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [8] U. Banerjee, T. S. Ukyab, and A. P. Chandrakasan, “Sapphire: A Configurable Crypto-Processor for Post-Quantum Lattice-based Protocols,” *IACR Trans. Cryptographic Hardware and Embedded Systems*, vol. 2019, no. 4, pp. 17–61, 2019.
- [9] Xilinx Inc., “Vivado Design Suite User Guide: Power Analysis and Optimization,” UG907 (v2020.1), 2020.